

Relazione Progetto

Metodologie di Programmazione

A.A. 2022/2023

Elena Palandri, matricola 7054433

Descrizione del progetto:

Il progetto java si occupa della gestione di una libreria, concentrandosi sulla scelta dei libri da esporre in vetrina e della creazione di uno scontrino.

Le classi principali sono articolo (Item), che può essere costituito da libri singoli (SingleBook) o da collane di libri (BookCollection) composte da più libri singoli. Ad esempio, la collana "Classici Europei" può essere composta dalle collane "Classici Italiani", "Classici Inglesi" e dai libri "Classico Francese" e "Classico Tedesco".

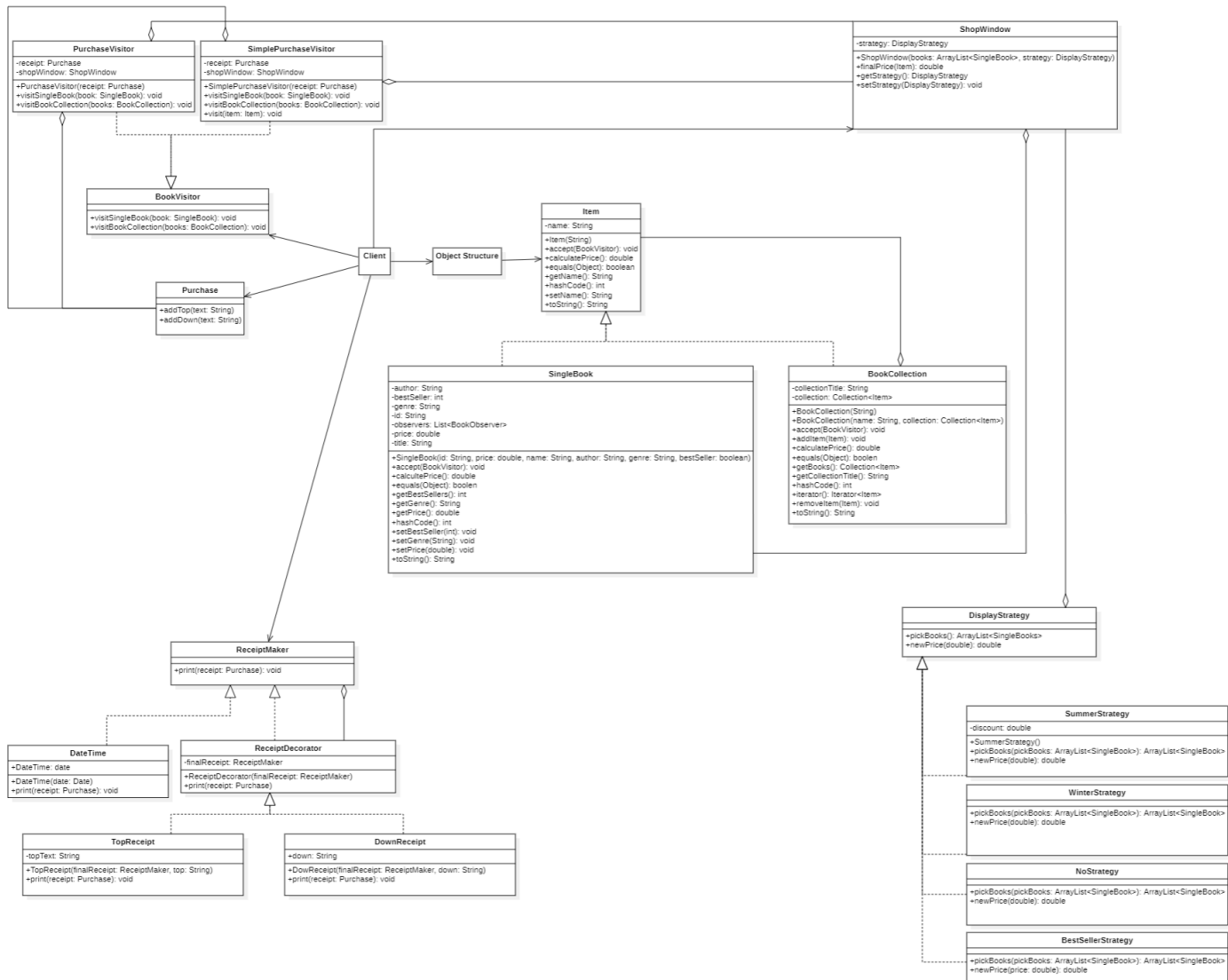
SingleBook è la classe più complessa, con più attributi e metodi in quanto ogni singolo libro dovrà essere descritto da diversi parametri per poterlo distinguere ed inoltre è l'unico che contiene il parametro prezzo, in quanto le collezioni sono sempre composte da più libri singoli e il prezzo è ricavabile attraverso un metodo. Una collezione di libri è messa in vendita al prezzo della somma del costo di ogni singolo libro, meno un 10% sul totale.

Ogni libro ha un costo che tendenzialmente rimane fisso, ma può essere soggetto a sconti se il libro inserito nella lista dei libri da esporre in vetrina, nella ShopWindow. I libri vengono selezionati in base a strategie diverse che scelgono i libri in base al genere letterario.

Il programma permette inoltre di creare uno scontrino che comprende tutti gli articoli acquistati. Vi sono due modalità di stampare lo scontrino, una che nel caso di acquisti di collane mostra gli articoli che lo compongono comprendendo tutti i singoli libri presenti, ed una più semplice che mostra solo il nome della collana. I libri acquistati singolarmente vengono visualizzati allo stesso modo per entrambe le modalità.

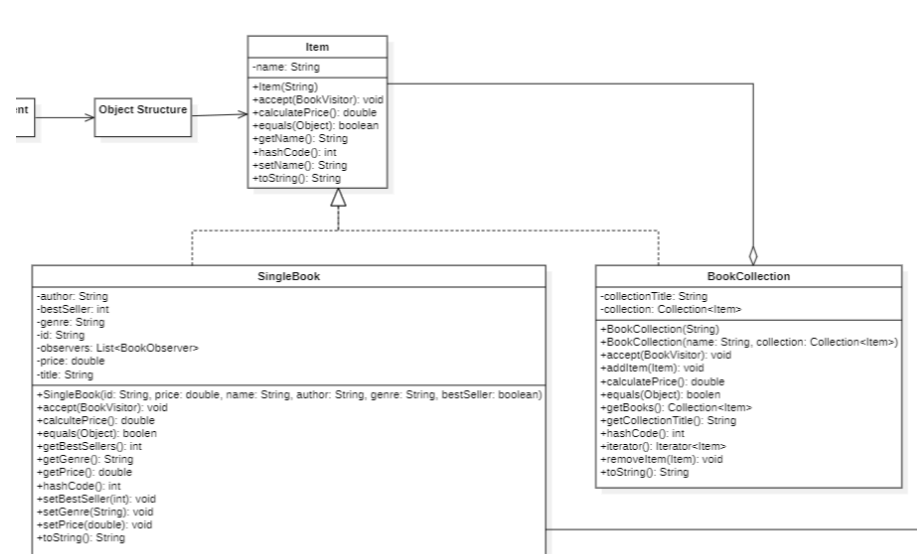
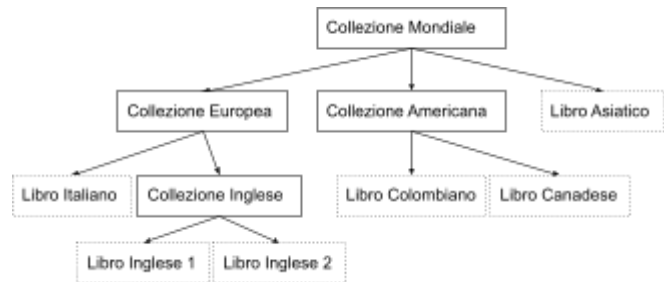
Nello scontrino, accanto al nome di ogni prodotto, è indicato il prezzo. Allo scontrino viene aggiunta un'intestazione con il nome e l'indirizzo del negozio e una parte finale che comprende i saluti, data e ora dell'acquisto.

Diagramma UML delle classi:

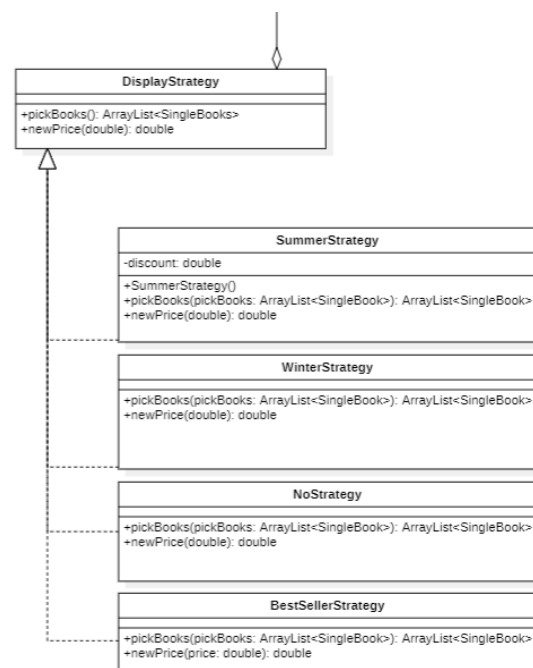


Pattern:

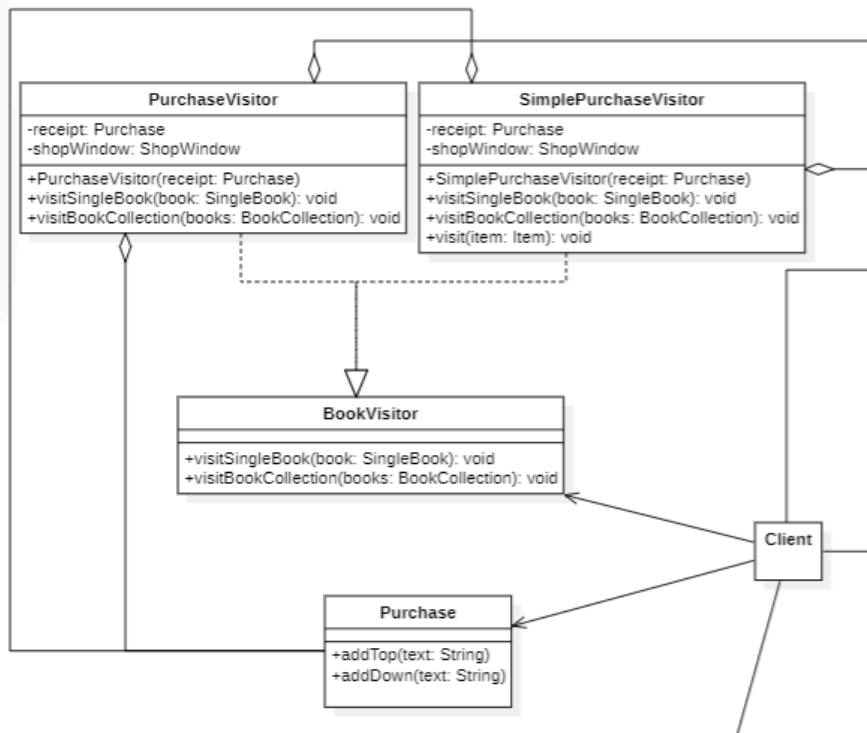
Composite: Il pattern organizza gli oggetti in una struttura ad albero.



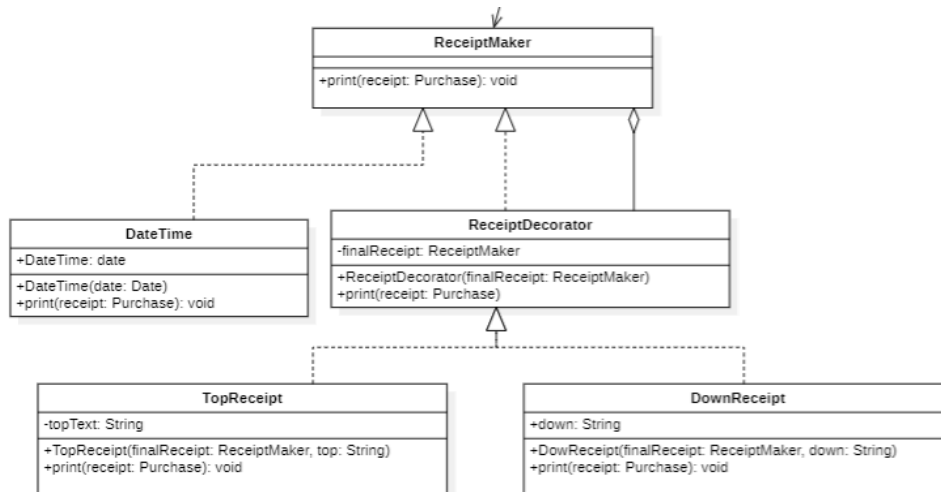
Strategy: Seleziona i libri da esporre nella vetrina del negozio e, se necessario, applica uno sconto al prezzo.



Visitor: Restituisce il prezzo effettivo di ogni prodotto da riportare nello scontrino, compresi eventuali sconti.



Decorator: Aggiunge informazioni all'inizio e alla fine dello scontrino.



Scelte di Design e Implementative:

Il progetto mira a gestire una libreria, concentrandosi sulla gestione della vetrina, dei prezzi e il loro calcolo per poi riportarli in uno scontrino.

Gli articoli in vendita sono libri singoli, collane di libri, costituite da più libri singoli, o composizioni di entrambe. La suddivisione è svolta con il pattern Composite, che organizza gli articoli in una struttura ad albero, nella quale i nodi rappresentano le BookCollection, e le foglie sono oggetti semplici, i SingleBook.

Il design del pattern è type safety, infatti le operazioni sui figli fanno parte solo dell'interfaccia Composite BookCollection. Quando creo un oggetto SingleBook o BookCollection, il compilatore è consapevole del tipo specifico. Ciò consente ad esso di controllare che le operazioni eseguite su ciascun oggetto siano coerenti con il suo tipo. Il client non può chiamare operazioni di gestione dei figli anche su una foglia, in questo modo si evitano errori a runtime.

La classe astratta Item ha come attributo solo il nome, mentre la classe BookCollection ha nome e una lista nella quale è possibile inserire tutti i libri da cui è composta.

SingleBook ha attributi id, titolo, autore, genere, prezzo e posto nella classifica Best Seller. Genere e BestSeller verranno utilizzati nell'applicazione delle strategie.

Il prezzo è riportato solo nella classe SingleBook, in quanto è possibile calcolare il costo delle collezioni e degli articoli tramite il metodo calculatePrice().

Il pattern Visitor permette di visitare ogni oggetto della collezione singolarmente.

Nella parte dei test, per verificare la correttezza del codice e che producesse i risultati attesi, ho usato una chiamata al metodo assert, in particolare assertEquals(actual, expected) che verifica se il risultato ottenuto, actual value, coincide con il risultato atteso, expected value.

Per testare il corretto funzionamento della classe SingleBook ho creato metodi che controllassero i valori del toString, hashCode e uno che verificasse se due libri fossero identici o meno. Per BookCollection ho fatto gli stessi test presenti in SingleBook ed inoltre ho testato la funzione calculatePrice per verificare che venisse calcolato il costo di ogni libro e che lo sconto sulla collezione venisse effettivamente applicato. Infine ho controllato che l'annidamento di due collezioni funzionasse correttamente, verificando che venissero riportati tutti i libri presenti nella collezione principale e nella sotto-collezione.

DisplayStrategy, a seconda della categoria del genere, seleziona libri col pattern Strategy che andranno esposti nella ShopWindow. Summer, Winter, BestSeller e NoStrategy sono le possibili modalità di selezione dei libri.

Per esempio, se siamo in periodo estivo i libri che hanno un genere Fantasy o Romance verranno selezionati per essere esposti nella shop window e verrà applicata una riduzione sul prezzo di partenza.

BestSeller invece seleziona i libri che rientrano nella categoria Horror e Mistero, impostano un valore in base alla loro posizione in classifica e applicano uno sconto. Dall'interfaccia ShopWindow è possibile ricavare il prezzo ridotto, se un libro è stato scontato prima di essere esposto. Il metodo finalPrice infatti prende il prezzo di un Item e, se necessario, applica la riduzione.

Questo pattern permette di selezionare e calcolare facilmente i prezzi in caso sia necessario applicare specifiche condizioni.

Per controllare il funzionamento delle classi, ho creato un campione di 10 libri potenzialmente esposti in vetrina ed ho verificato che ogni strategia prendesse solo i libri che corrispondenti alle loro richieste.

Per creare lo scontrino è necessario avere la lista di tutti i libri o collane e del loro prezzo. L'interfaccia Purchase permette di aggiungere in cima o in fondo informazioni che poi verranno utilizzate nello scontrino. Le classi PurchaseVisitor e SimplePurchaseVisitor implementano l'interfaccia BookVisitor. Questa interfaccia dichiara due metodi, visitSingleBook e visitBookCollection, che vengono implementati nelle classi visitatrici.

PurchaseVisitor genera una ricevuta basata su libri singoli e collezioni di libri. Il metodo visitSingleBook aggiunge il prezzo finale di un libro singolo alla ricevuta, mentre visitBookCollection aggiunge quello di una raccolta di libri, mostrando anche ogni singolo libro ed il prezzo con applicato uno sconto del 10%. Per riuscire a visualizzare tutti i libri presenti ho utilizzato Iterator.

SimplePurchaseVisitor è una versione semplificata di quella appena descritta e produce un elenco meno dettagliato, nel quale la collezione di libri è riportata senza specificare tutti i libri che la compongono.

Per il corretto sviluppo del pattern ho aggiunto anche i metodi accept in Item e nelle sottoclassi.

Per testare il visitor e le funzionalità delle classi ho creato un'implementazione fittizia ("mock") nella directory dei test MockMakeReceipt che usa uno stringBuilder per gestire le operazioni di scrittura aggiuntiva all'inizio o alla fine della ricevuta.

Ho creato anche due classi per testare le due implementazioni. PurchaseVisitorTest verifica che le informazioni riportate nello scontrino dopo aver chiamato il visitor siano corrette. L'ho testato con libri singoli, collane di libri e un insieme di una collezione costituita da 2 collezioni e un libro singolo, per assicurarmi che lo scontrino stampasse tutto come desiderato, in particolare mostrando tutti i libri singoli appartenenti alle collezioni.

SimplePurchaseVisitor esegue gli stessi test descritti prima, per le ricevute di libri singoli si comporta allo stesso modo, ma per le collezioni stampa solo esclusivamente il titolo e il prezzo scontato, senza ulteriori dettagli. Ho inoltre testato

che un libro nella lista dei libri esposti nella vetrina estiva riportasse il prezzo corretto per verificare che il pattern strategy fosse applicato correttamente.

Infine, per rendere la ricevuta più verosimile ho deciso di aggiungere delle stringhe in cima e alla fine dello scontrino.

ReceiptMaker è un'interfaccia che dichiara il metodo `print`, implementato da tutte le classi che vogliono creare la ricevuta. ReceiptDecorator è una classe astratta che implementa l'interfaccia ReceiptMaker. Il campo `finalReceipt` rappresenta la ricevuta finale a cui verranno aggiunti ulteriori dettagli. Il metodo `print` viene ereditato dall'interfaccia e viene implementato per chiamare il metodo sulla ricevuta finale.

DateTime implementa l'interfaccia ReceiptMaker e nel campo `dateTime` memorizza la data e l'ora. Il metodo `print` stampa la data e l'ora nella ricevuta.

TopReceipt è una delle classi che estende ReceiptDecorator. Il campo `topText` rappresenta il testo che verrà aggiunto nella parte superiore della ricevuta. Il metodo `print` aggiunge il testo alla parte superiore della ricevuta e quindi chiama il metodo sulla ricevuta finale.

DownReceipt estende ReceiptDecorator. Il campo `downText` rappresenta il testo che verrà aggiunto nella parte inferiore della ricevuta. Il metodo `print` si occupa di aggiungere il testo alla parte inferiore della ricevuta e, come TopReceipt chiama `print` sulla ricevuta finale.

Questo sistema consente di comporre diverse funzionalità di stampa (data, testo in alto, testo in basso) in modo flessibile, permettendo di creare una ricevuta personalizzata con combinazioni di queste caratteristiche.

Per testare il pattern decorator ho creato `makeDateReceiptTest` che verifica la stampa di data e ora. `MakeTopReceiptTest` verifica che il testo aggiunto in alto corrisponda a "Bookshop 1st Avenue 1234" e `makeDownReceipt` che il testo nella parte finale sia "Thanks for the purchase!".

Il test `makeReceipt` testa interamente la creazione delle aggiunte allo scontrino. Crea un'istanza di TopReceipt che decora un oggetto DownReceipt che a sua volta decora un oggetto DateTime e stampa la ricevuta. Successivamente verifica se la stringa risultante sulla ricevuta rispecchia le aspettative, ovvero contiene il testo in alto, la sezione dedicata agli articoli acquistati, seguito dal testo in basso e dalla data.